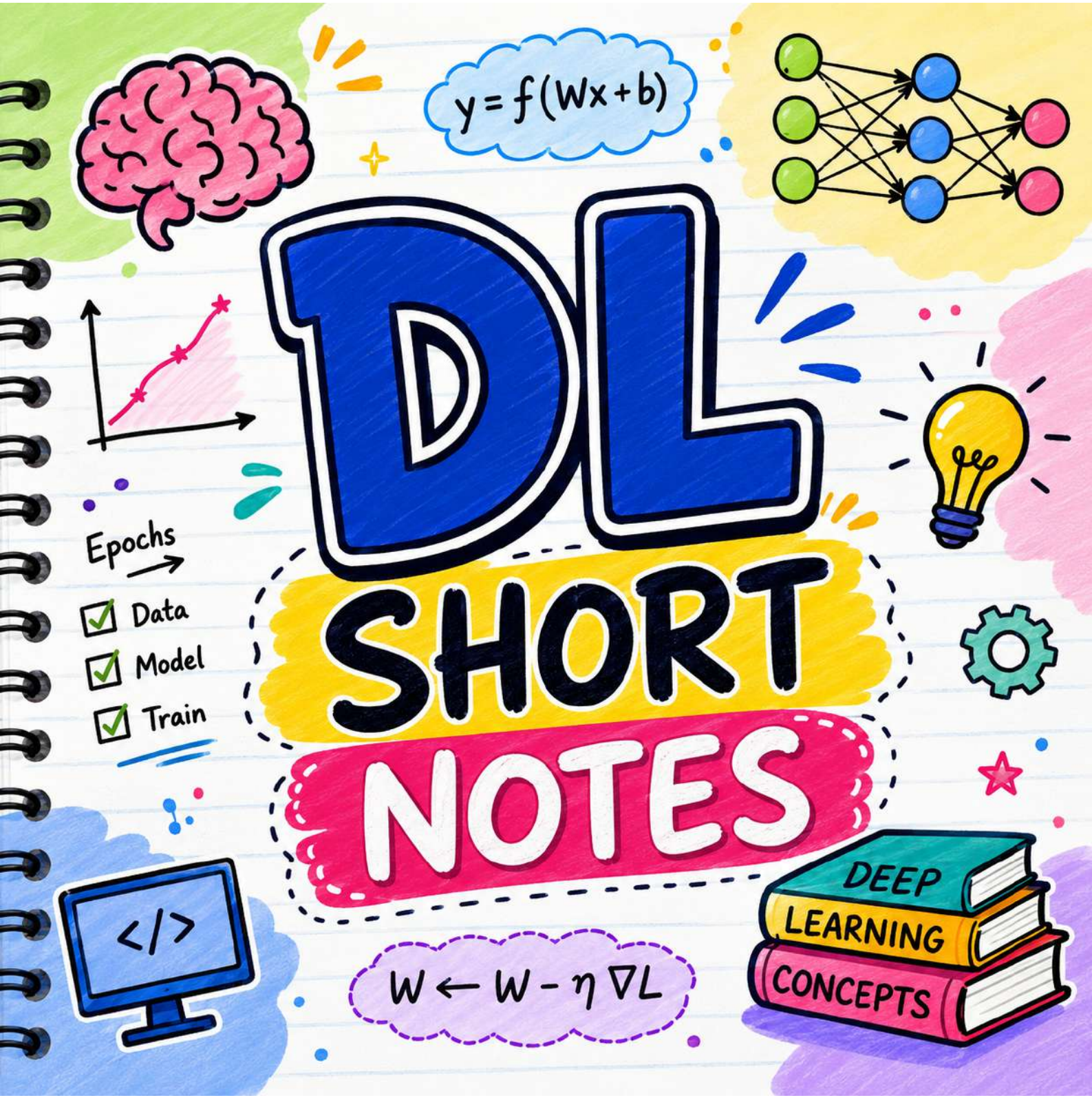

y = f(Wx + b)
DL
SHORT
NOTES
Epochs
Data
Model
Train
DEEP
LEARNING
CONCEPTS
W ← W - η ∇L

DEEP LEARNING – COMPLETE NOTES

1 INTRODUCTION TO DEEP LEARNING

- Deep Learning is a subfield of Machine Learning that uses artificial neural networks with multiple layers (hence "deep") to model and understand complex patterns in data.
- It has achieved state-of-the-art results in areas such as computer vision, natural language processing, speech recognition, robotics, and more.
- Deep Learning models learn hierarchical representations: lower layers learn simple features, while higher layers learn more abstract features.

1.1 WHY DEEP LEARNING?

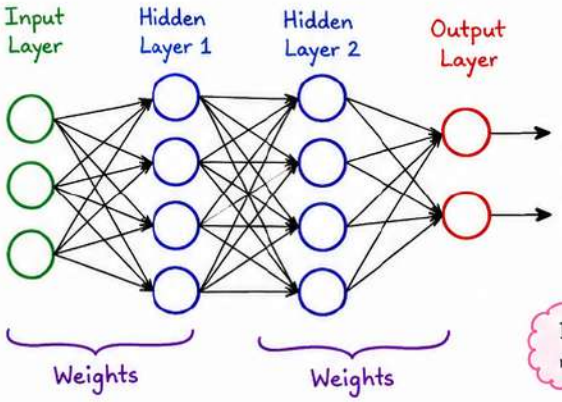
- ✓ Automatically learns features from raw data → No manual feature engineering
- ✓ Handles large amounts of data and high dimensionality → Better performance
- ✓ Works well for unstructured data (images, audio, text)
- ✓ Scales with data and computation

1.2 DIFFERENCE: MACHINE LEARNING vs DEEP LEARNING

Aspect	Machine Learning	Deep Learning
Feature Engineering	Manual	Automatic (learned)
Data Requirement	Works with small data	Requires large data
Model Complexity	Shallow models	Deep (many layers)
Performance	Lower on complex tasks	Higher on complex tasks
Examples / Algorithms	SVM, Decision Tree, Logistic Reg.	NN, CNN, RNN, Transformers

1.3 WHAT IS A NEURAL NETWORK?

- A Neural Network (NN) is a computational model inspired by the human brain.
- It consists of interconnected neurons (or nodes) organized in layers.
- Each connection has a weight, and each neuron applies an activation function.



- Input Layer: receives the raw data.
- Hidden Layers: learn intermediate & abstract features.
- Output Layer: produces the final prediction.

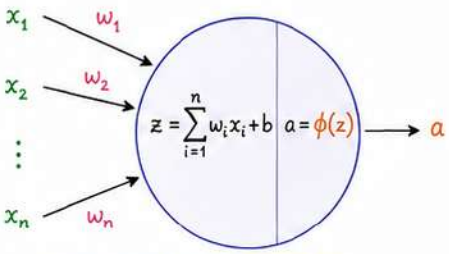
Deeper networks can learn more complex patterns!

DEEP LEARNING – COMPLETE NOTES

2. NEURON, ACTIVATIONS & LAYERS

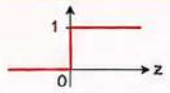
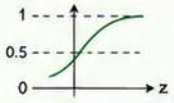
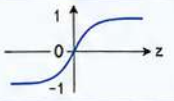
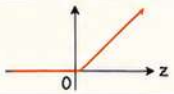
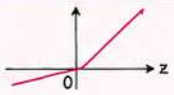
2.1 Neuron (Perceptron)

A neuron computes a weighted sum of inputs, adds bias and applies an activation function.



Where,
- x_i : inputs
- w_i : weights
- b : bias
- ϕ : activation function

2.2 Common Activation Functions

Name	Formula	Graph
Step	$f(z) = \begin{cases} 1, & z \geq 0 \\ 0, & z < 0 \end{cases}$	
Sigmoid	$f(z) = \frac{1}{1+e^{-z}}$	
tanh	$f(z) = \frac{e^{z} - e^{-z}}{e^{z} + e^{-z}}$	
ReLU	$f(z) = \max(0, z)$	
Leaky ReLU	$f(z) = \begin{cases} z, & z > 0 \\ \alpha z, & z \leq 0 \end{cases}$ (α small, e.g., 0.01)	

2.3 Types of Layers

Input Layer
Receives raw data.

Dense (Fully Connected) Layer
Every neuron connected to all neurons in previous layer.

Convolutional Layer (Conv)
Applies filters/kernels to local regions (good for images).

Pooling Layer
Reduces spatial size (e.g., Max Pool, Avg Pool).

Dropout Layer
Randomly drops neurons during training to prevent overfitting.

Output Layer
Produces final output (e.g., class probabilities).

3. LOSS FUNCTIONS

Loss function measures how far the prediction is from the true target. The goal is to minimize the loss.

3.1 For Regression

Mean Squared Error (MSE)

$$L = \frac{1}{n} \sum_{i=1}^{n} (y_i - \hat{y}_i)^2$$

Where, y_i = true value, $\hat{y}_i$ = predicted value

Mean Absolute Error (MAE)

$$L = \frac{1}{n} \sum_{i=1}^{n} |y_i - \hat{y}_i|$$

3.2 For Classification

Binary Cross-Entropy (Log Loss)

$$L = -\frac{1}{n} \sum_{i=1}^{n} \left[y_i \log(\hat{y}_i) + (1-y_i) \log(1-\hat{y}_i) \right]$$

$y_i \in \{0, 1\}$

Categorical Cross-Entropy (Multi-class)

$$L = -\frac{1}{n} \sum_{i=1}^{n} \sum_{k=1}^{K} y_{ik} \log(\hat{y}_{ik})$$

K = number of classes

NOTE

- Smaller loss → better model
- Choice of loss depends on the problem (regression vs classification)
- We minimize the loss during training using an optimizer.

4. OPTIMIZATION: GRADIENT DESCENT

We update the parameters (weights and bias) to minimize the loss. Gradient Descent updates:

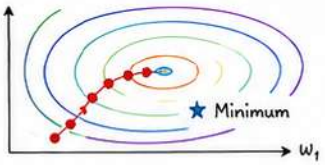
$$\theta_{new} = \theta_{old} - \eta \frac{\partial L}{\partial \theta}$$

- θ : parameter (w or b)
- η : learning rate
- $\frac{\partial L}{\partial \theta}$: gradient of loss w.r.t. parameter

Types of Gradient Descent

- Batch GD : uses entire dataset
- Stochastic GD : uses one example at a time
- Mini-batch GD : uses small batch (e.g., 32, 64)

★ Learning rate (η) controls the step size. Too high → overshoot, Too low → slow learning.



Goal : reach the minimum of the loss surface.

DEEP LEARNING – COMPLETE NOTES

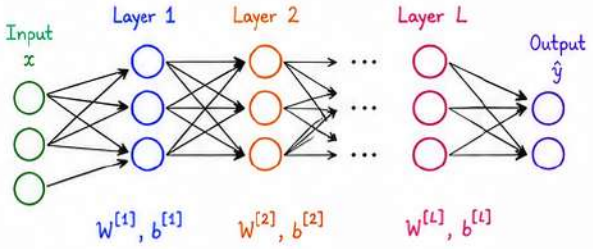
5. FORWARD PROPAGATION

Forward propagation computes the output of the network given an input by passing it through all layers.

For a layer ℓ:

$$z^{[\ell]} = W^{[\ell]} a^{[\ell-1]} + b^{[\ell]} \quad \leftarrow \text{(linear step)}$$

$$a^{[\ell]} = g^{[\ell]}(z^{[\ell]}) \quad \leftarrow \text{(activation step)}$$

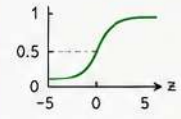


z : linear combination (pre-activation) a : activation (post-activation) g : activation function

6. COMMON ACTIVATION FUNCTIONS

Sigmoid

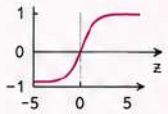
$$\sigma(z) = \frac{1}{1+e^{-z}}$$



- Output in (0, 1)
- Not zero-centered

tanh

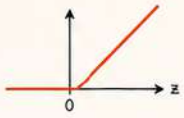
$$g(z) = \tanh(z)$$



- Output in (−1, 1)
- Zero-centered

ReLU

$$g(z) = \max(0, z)$$

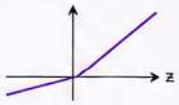


- Simple & effective
- Risk of "dying ReLU"

Leaky ReLU

$$g(z) = \max(\alpha z, z)$$

(α small, e.g., 0.01)



- Allows small gradient
- Mitigates dying ReLU

Softmax

$$y_i = \frac{e^{z_i}}{\sum_{j=1}^{K} e^{z_j}}$$

- Converts scores to probabilities
- Used in output layer for multi-class classification

7. REGULARIZATION

Regularization helps prevent overfitting and improves generalization.

7.1 L2 Regularization (Weight Decay)

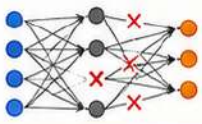
Adds a penalty on large weights.

$$\text{Loss: } L = L_{data} + \lambda \sum_{\ell} \|W^{[\ell]}\|_2^2$$

λ : regularization strength

7.2 Dropout

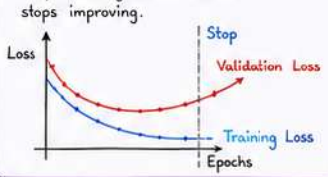
Randomly drops neurons during training with probability p.



Keep probability = $1 - p$

7.3 Early Stopping

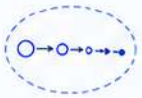
Stop training when validation loss stops improving.



8. VANISHING & EXPLODING GRADIENTS

Vanishing Gradients

- Gradients become very small as they backpropagate.
- Weights update very slowly.
- Common in deep networks with sigmoid/tanh.

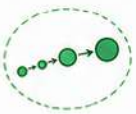


Mitigation: Use ReLU, proper initialization, Batch Normalization, Skip Connections.

Backpropagation computes gradients from output to input using the chain rule. If gradients are too small or too large, learning becomes ineffective.

Exploding Gradients

- Gradients grow exponentially.
- Weights update wildly.
- Can make training diverge.



Mitigation: Gradient Clipping, proper initialization, normalization.

Takeaway: Choose suitable activations, regularize your model, and monitor gradients for stable and efficient training.

DEEP LEARNING – COMPLETE NOTES

9. INITIALIZATION TECHNIQUES

Good initialization helps in faster convergence and prevents vanishing / exploding gradients.

- **Zero Initialization**

  $W = 0$ (Not recommended)

- **Random Initialization**

  $W \sim N(0, \sigma^2)$ (σ small)

- **Xavier / Glorot Initialization**

$$\text{Var}(W) = \frac{2}{n_{in} + n_{out}}$$

- **He Initialization (for ReLU)**

$$\text{Var}(W) = \frac{2}{n_{in}}$$

- **LeCun Initialization (for SELU)**

$$\text{Var}(W) = \frac{1}{n_{in}}$$

Where, n_{in} = number of input
n_{out} = number of output units of the layer

10. NORMALIZATION TECHNIQUES

Normalization stabilizes and accelerates training.

10.1 Batch Normalization (BN)

Compute mini-batch mean μ_B and variance σ_B^2 → Normalize the activations $\hat{x} = \frac{x - \mu_B}{\sqrt{\sigma_B^2 + \epsilon}}$ → Scale and shift $y = \gamma \hat{x} + \beta$ → γ, β are learnable parameters

10.2 Layer Normalization (LN)

Normalize across features in a single example (good for RNNs / Transformers).

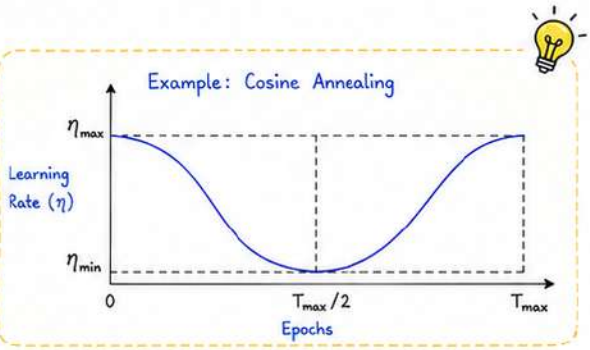
10.3 Group Normalization (GN)

Normalize across groups of channels (works well for small batch sizes).

11. LEARNING RATE SCHEDULING

Adjust the learning rate (η) during training for better convergence.

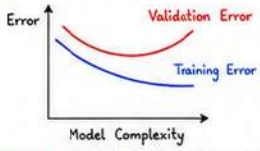
Scheduler	Formula / Rule	Description
Step Decay	$\eta_t = \eta_0 \times \gamma^{[t/k]}$	Reduce η by factor γ every k epochs
Exponential Decay	$\eta_t = \eta_0 \times \gamma^t$	Smooth exponential decrease
Cosine Annealing	$\eta_t = \eta_{min} + \frac{1}{2}(\eta_{max} - \eta_{min})\left(1 + \cos\left(\frac{\pi t}{T_{max}}\right)\right)$	Cosine curve from η_{max} to η_{min}
Reduce on Plateau	If metric not improved for P epochs → $\eta = \eta \times \gamma$	Reduce when progress stalls



12. OVERFITTING, UNDERFITTING & GENERALIZATION

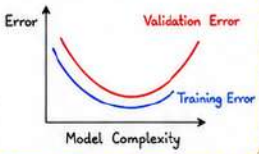
Underfitting (High Bias)

- Model too simple
- Can't capture underlying pattern
- High training & high validation error



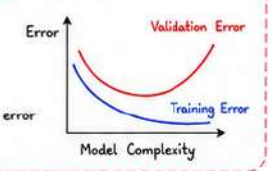
Good Fit (Good Generalization)

- Model captures the pattern well
- Low training & low validation error



Overfitting (High Variance)

- Model too complex
- Learns noise
- Low training error but high validation error



13. COMMON METRICS

Accuracy

$$\frac{\text{\# Correct}}{\text{\# Total}}$$

Precision

$$\frac{TP}{TP + FP}$$

Recall (Sensitivity)

$$\frac{TP}{TP + FN}$$

F1-Score

$$\frac{2 \times \text{Precision} \times \text{Recall}}{\text{Precision} + \text{Recall}}$$

ROC-AUC

Area Under ROC Curve

Where,
TP : True Positive
FP : False Positive
TN : True Negative
FN : False Negative

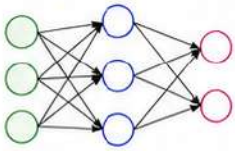
Master the fundamentals. Practice consistently. Build amazing models!

DEEP LEARNING - COMPLETE NOTES

14. NEURAL NETWORK ARCHITECTURES

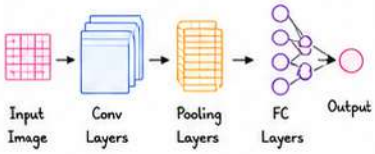
Different architectures are designed for different types of data and tasks.

1. Feedforward Neural Network (FNN)



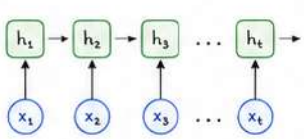
- Information flows only forward.
- No cycles or loops.
- Used for simple tasks like regression, classification.

2. Convolutional Neural Network (CNN)



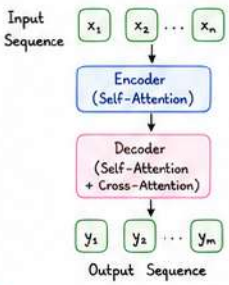
- Uses convolution and pooling.
- Captures spatial hierarchies.
- Best for image and video data.

3. Recurrent Neural Network (RNN)



- Has loops (cycles).
- Maintains hidden state (memory).
- Used for sequences (text, speech, time series).

4. Transformer (Attention-based)



- No recurrence; uses attention.
- Captures long-range dependencies.
- State-of-the-art in NLP.

15. CONVOLUTION OPERATION (CNN)

Convolution extracts local patterns using a small filter (kernel) that slides over the input.

Example:

Input (5×5)

1	2	1	0	1
0	1	2	1	0
1	0	1	2	1
0	1	0	1	2
1	2	1	0	1

*

Kernel / Filter (3×3)

1	0	-1
1	0	-1
1	0	-1

→

Feature Map (3×3)

-2	0	-2
-2	0	-2
-2	0	-2

How it works:

1. Place the kernel on the top-left of the input.
2. Element-wise multiply and sum the values → one output.
3. Slide the kernel by stride and repeat.
4. Produces a feature map.

Key Hyperparameters

- Filter size (e.g., 3×3, 5×5)
- Stride (e.g., 1, 2)
- Padding (same / valid)
- Number of filters (depth)

CNNs automatically learn useful filters during training.

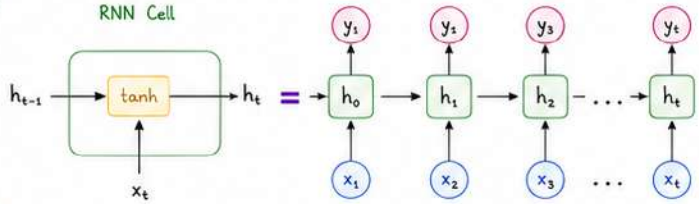
16. RECURRENT NEURAL NETWORKS (RNN)

RNNs process sequential data by maintaining a hidden state that captures information from previous time steps.

Use Cases

- Language modeling
- Machine translation
- Sentiment analysis
- Time series forecasting

Unrolled RNN



Types of RNN Cells

- Simple RNN
- LSTM (Long Short-Term Memory)
- GRU (Gated Recurrent Unit)

Problem with Simple RNNs

- Struggle with long-term dependencies (vanishing / exploding gradients).
- LSTM / GRU solve this using gates.

Quick Comparison

Model	FNN	CNN	RNN	Transformer
Best For	Tabular data, basic tasks	Images, videos	Sequences, time series	Sequences (NLP), long context
Key Idea	Fully connected layers	Local patterns + weight sharing	Memory through hidden state	Attention mechanism
Strength	Simple & fast	Captures spatial info	Handles variable lengths	Captures long-range dependencies
Limitation	Ignores structure	Needs more data/compute	Slow to train, vanishing gradients	High compute & memory

Different architectures, same goal: Learn representations that solve real-world problems!

DEEP LEARNING – COMPLETE NOTES

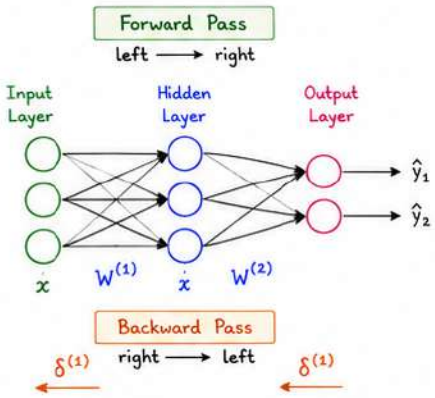
17. BACKPROPAGATION ALGORITHM

Backpropagation efficiently computes gradients by applying the chain rule from output to input.

Steps:

1. **Forward Pass:** Compute activations layer by layer.
2. **Compute Loss:** Compare output with target.
3. **Backward Pass:** Compute gradients from output layer to input layer.
4. **Update Weights:** Use optimizer (e.g., GD, Adam).

Key Idea:
Reuse intermediate results to compute gradients efficiently.



Gradient for weight $w_{ij}^{(l)}$

$$\frac{\partial L}{\partial w_{ij}^{(l)}} = a_i^{(l-1)} \delta_j^{(l)}$$

Where,
- $a_i^{(l-1)}$: activation from previous layer
- $\delta_j^{(l)}$: error term (delta) of current layer

Common Output Layer Deltas

- Regression (MSE): $\delta = (\hat{y} - y) \odot f'(z)$
- Binary Cross-Entropy: $\delta = (\hat{y} - y)$
- Softmax + Cross-Entropy: $\delta = (\hat{y} - y)$

18. OPTIMIZERS

Optimizers update weights to minimize the loss.

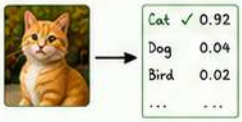
Optimizer	Update Rule ($\theta_t \to \theta_{t+1}$)	Key Idea	Pros	Cons
SGD (Stochastic GD)	$\theta_{t+1} = \theta_t - \eta \nabla L_t$	Uses one example at a time	Simple, memory efficient	Noisy updates, slower convergence
Momentum	$v_t = \beta v_{t-1} + \nabla L_t$ $\theta_{t+1} = \theta_t - \eta v_t$	Adds momentum to smooth updates	Reduces oscillations, faster	Needs tuning (β)
RMSProp	$s_t = \beta s_{t-1} + (1-\beta)(\nabla L_t)^2$ $\theta_{t+1} = \theta_t - \eta \frac{\nabla L_t}{\sqrt{s_t} + \epsilon}$	Adapts learning rate per parameter	Good for non-stationary problems (RNNs)	Can decay learning rate too much
Adam (Most popular)	$m_t = \beta_1 m_{t-1} + (1-\beta_1)\nabla L_t$ $v_t = \beta_2 v_{t-1} + (1-\beta_2)(\nabla L_t)^2$ $\theta_{t+1} = \theta_t - \eta \frac{\hat{m}_t}{\sqrt{\hat{v}_t} + \epsilon}$	Combines momentum + RMSProp (adaptive)	Fast, efficient, works well in practice	More hyperparameters

Where,
- η : learning rate
- β : decay rate ($0 < \beta < 1$)
- ϵ : small constant to avoid division by zero

Tip:
Adam with default settings (η=0.001, β_1=0.9, β_2=0.999, ϵ=10^{-8}) works well in most cases!

19. OVERVIEW OF COMMON TASKS IN DL

1. Image Classification
Assign a label to the entire image.



Example: Cat vs Dog

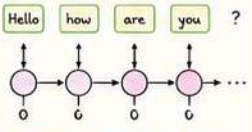
2. Object Detection
Detect and locate multiple objects in an image.

Example: YOLO, SSD, Faster R-CNN

3. Semantic Segmentation
Classify each pixel in the image.

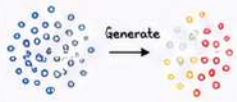
Example: U-Net, DeepLab

4. Sequence Modeling
Process sequence data (step by step).



Example: RNN, LSTM, GRU

5. Generative Modeling
Generate new data similar to training data.



Example: GANs, VAEs

20. TIPS TO IMPROVE YOUR MODELS

- **More Data** – More diverse data usually helps.
- **Data Preprocessing** – Clean, normalize, and augment data.
- **Tune Hyperparameters** – Learning rate, batch size, layers, neurons, etc.
- **Regularization** – Dropout, L2, BatchNorm to reduce overfitting.
- **Monitor & Evaluate** – Use validation set, track metrics and curves.
- **Experiment & Iterate** – Small changes → Big improvements!

Deep learning is a journey of continuous learning, experimentation and persistence. Keep building!

DEEP LEARNING – COMPLETE NOTES

21. COMMON CHALLENGES

1. Overfitting
- Model learns noise in training data.
- Fix: More data, Regularization, Dropout, Early stopping.

2. Underfitting
- Model too simple to learn pattern.
- Fix: Bigger model, More features, Train longer.

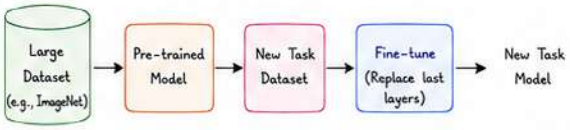
3. Vanishing / Exploding Gradients
- Gradients become too small or too large in deep networks.
- Fix: ReLU, Xavier/He init, Gradient clipping, BatchNorm.

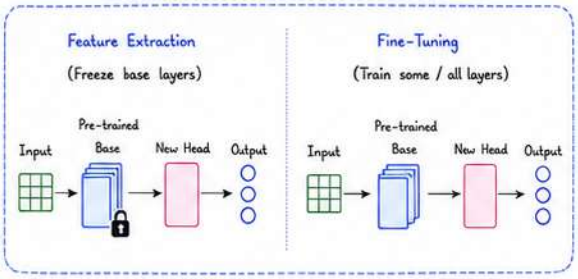
4. Data Issues
- Insufficient / imbalanced / noisy data.
- Fix: Data augmentation, Resampling, Cleaning.

5. High Computational Cost
- Training deep models is expensive.
- Fix: Better hardware, Mixed precision, Efficient models.

22. TRANSFER LEARNING & FINE-TUNING

Use a pre-trained model on a large dataset and adapt it to a new task.





23. ACTIVATION SUMMARY

Activation	Range	Pros	Cons
Sigmoid	(0, 1)	Smooth, Probabilities	Vanishing gradients
tanh	(-1, 1)	Zero-centered	Vanishing gradients
ReLU	[0, ∞)	Fast, Sparse activation	Dying ReLU problem
Leaky ReLU	(-∞, ∞)	Fixes dying ReLU	Hyperparameter (α)
Softmax	(0, 1) (sum=1)	Multi-class output	Not for hidden layers

24. COMMON DL FRAMEWORKS

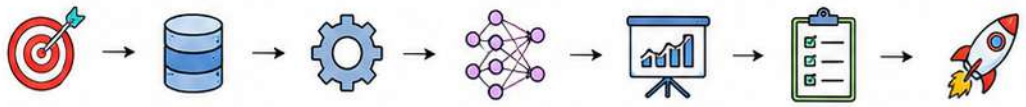
TensorFlow
- Developed by Google
- Flexible, scalable
- Keras API is easy to use

PyTorch
- Developed by Facebook (Meta)
- Dynamic computation graph
- Great for research

Keras
- High-level API
- Runs on top of TF / PyTorch
- Beginner friendly

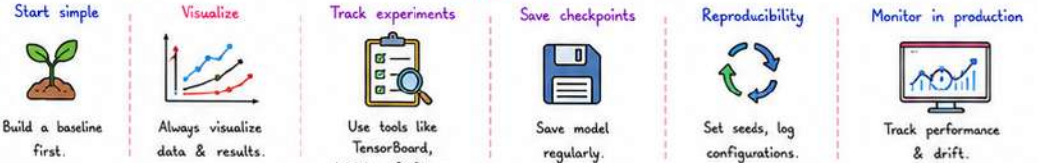
JAX
- High performance
- Auto-diff + XLA compiler
- Great for scientific ML

25. DEEP LEARNING MODEL LIFE CYCLE



Understand goal, metrics, constraints. | Gather, label and prepare data. | Clean, normalize, split train/val/test. | Choose architecture, layers, activations. | Train, validate, monitor metrics. | Test on unseen data, analyze errors. | Deploy to production, monitor performance.

Best Practices



26. CHEAT SHEET – KEY FORMULAS

- Linear: $z = Wx + b$
- Activation: $a = g(z)$
- MSE Loss: $L = \frac{1}{n}\sum_{i=1}^{n}(y_i - \hat{y}_i)^2$
- Cross-Entropy (Binary): $L = -\frac{1}{n}\sum_{i=1}^{n}[y_i \log(\hat{y}_i) + (1 - y_i)\log(1 - \hat{y}_i)]$
- Softmax: $y_i = \frac{e^{z_i}}{\sum_{j=1}^{K} e^{z_j}}$
- Gradient Descent: $\theta_{new} = \theta_{old} - \eta \frac{\partial L}{\partial \theta}$
- Backprop: $\frac{\partial L}{\partial \theta} = \frac{\partial L}{\partial z} \cdot \frac{\partial z}{\partial \theta}$ (chain rule)

27. FINAL TAKEAWAYS

- Deep learning learns hierarchical representations automatically.
- Data + Model + Compute are the 3 pillars of success.
- Start simple, iterate, and keep improving.
- Understand the basics deeply – it helps in everything!
- Practice consistently and build real-world projects.

Keep Learning. Keep Building. The more you practice, the better you become! ♡